RailFuse ContextStore

面向版本化 KV Cache 对象的故障原子多轨读取引擎

技术设计、接口契约、真实 RXE 验证与可复现实验报告

以更少的内存搬运，交付可验证的完整对象

一次对象读取需要同时回答：怎样分配路径，何时发布结果，何时回收内存。作品沿用 DaoCloud ContextStore 的服务、协议与条带布局，扩展客户端读取契约，并通过真实 Verbs/RXE 的失败反例验证异步错误后的缓冲区复用与恢复。

问题	机制	可复核证据
安全暂存有额外组装开销	有条件直接写入私有有序暂存；保留最终发布复制	同调度 10 个配对进程块：64 / 256 MiB 分别 1.100× / 1.161×
读取中发生对象覆盖	传输后复查完整对象身份与 Placement	20 次读中 8 次明确观察到 Generation 增长并拒绝发布
错误返回后仍有 DMA / 旧完成	先退休 QP、排空 CQ，再复用与重连	旧库精确污染负对照；修复版同 MR 哨兵、GET / PUT 恢复

阅读方式

第 1-3 章说明项目与边界，第 4-13 章解释实现与取舍，第 14-19 章报告分层证据，第 20 章给出部署与下一步验证。附录提供配置、公共接口和可复查命令，末尾列出 32 个源码及证据入口。

环境边界：当前实测包含真实软件 RoCE / RXE；没有物理双 HCA 聚合或端到端推理增益的测量。历史实验按原始日期分别报告。本次 V3 只更新材料和配音，未修改核心运行时。

核心安全提交：39ce2d50fcee00405758f1e2c4f7bd389baccf1c

代码与最新材料入口

目录

页码与书签可点击跳转；章节连续排版，图、表和代码紧随相应说明。

01 项目目标：把一次并行传输变成可交付的完整对象 3

02 应用场景与需求：对象、版本和内存有不同的完成时刻 4

03 系统边界与上游归属：增量落在客户端读取契约 4

04 元数据契约：先证明条带描述自洽，再打开数据连接 5

05 路径模型与发现：保持存储归属，显式选择网络 6

06 条带调度：按已分配字节平衡，故障后重建下一次计划 7

07 直接有序暂存：以互斥区域代替完整对象组装 7

08 SGE 映射与服务端回退：数据位置不能随着快慢路径改变 8

09 资源预算：请求结束与暂存释放分两次结算 9

10 发布契约：完整性、二次身份查询和取消共用一道门 10

11 Rust DMA 生命周期：先退休连接，再解除注册 10

12 Python 与 C ABI：明确单连接和外部内存责任 11

13 故障分类与恢复：错误名称要指导下一次动作 12

14 验证体系：每层证据都对应一个可观察契约 13

15 晚到 WRITE 负对照：同一注册区必须留到终态观察之后 14

16 PUT 与旧 CQE：重连成功必须包含真实完成归属 15

17 公平基线与负结果：更快的一轨改变了多轨结论 15

18 受控性能实验：固定计划，测量直接暂存的净效果 16

19 Generation 切换见证：复查版本确实在真实执行中阻止发布 18

20 部署与验证路线：先可复现契约，再判断硬件和业务收益 19

附录 A：Linux 构建与演示前置检查 20

附录 B：已记录双 RXE fixture 的关键配置 20

附录 C：Rust 公共 API 的完整对象读取 22

附录 D：Python 单连接的安全使用与边界 23

附录 E：演示、历史实验与审计复算入口 23

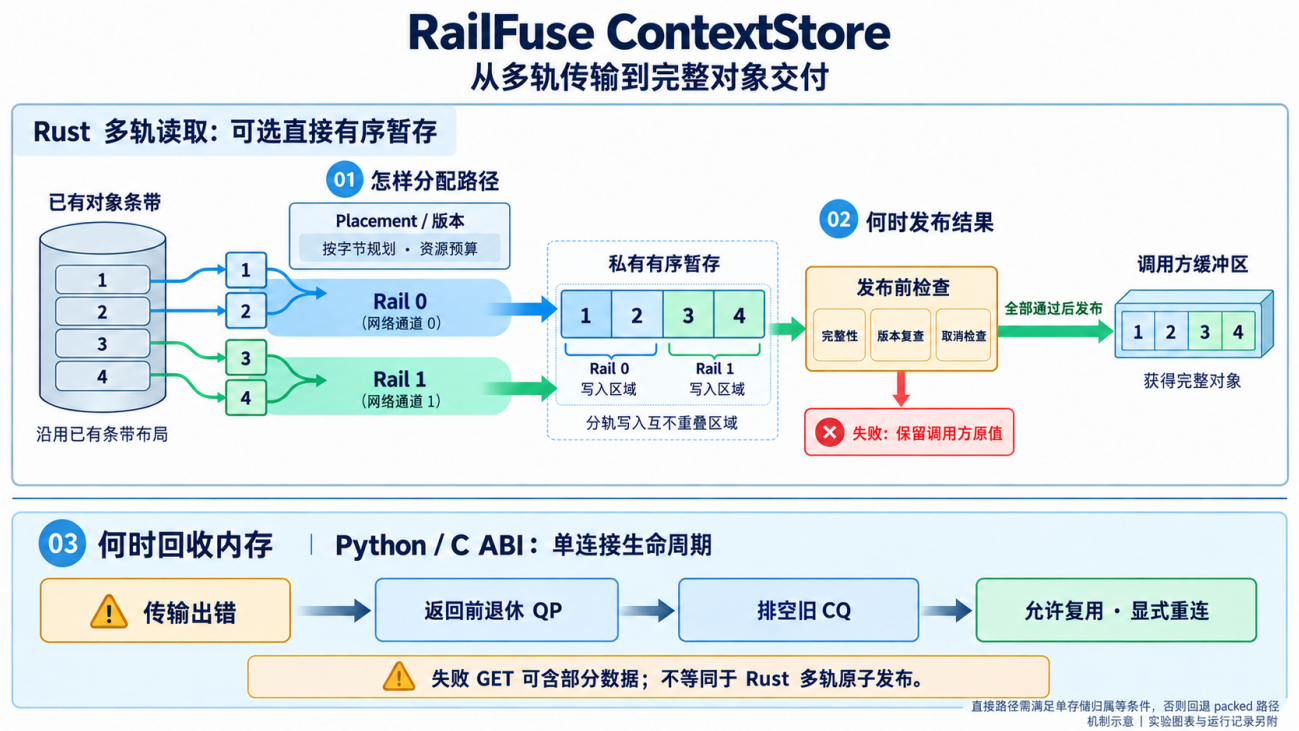
附录 F：证据分层与阅读索引 24

源码与证据索引 25

01 项目目标：把一次并行传输变成可交付的完整对象

RailFuse 基于 DaoCloud ContextStore，处理单个推理 Worker 读取共享 KV Cache 对象时的数据路径问题。对象仍保存在原有磁盘条带中；客户端依据对象描述符和实际 Placement，把属于同一存储节点的条带分配给多条 RDMA 路径。设计希望同时做到三件事：各轨只读取计划内的字节，调用方只接收检查通过的完整对象，失败后先终止相关 DMA 所有权再允许内存复用。

这些目标来自同一个具体冲突。共享存储可能利用多个磁盘或多个网络接口提供数据，但并行连接本身只解决传输组织。只要某条路径失败、对象在读取期间被覆盖，或者服务端 WRITE 在客户端错误返回后才到达，直接接收的应用缓冲区就可能混有部分结果或旧请求数据。向上层返回一个错误码，并不能自动撤回已经发生的写入，也不能证明旧 QP 已经不能访问这块内存。



总体机制示意：对象计划、私有暂存、检查发布与 DMA 退休。生成式机制图用于解释结构；数值结论以审计 JSON 和运行收据为准。

Rust SDK 采用请求私有暂存：先接收、再核验、最后统一复制到调用方。直接有序暂存进一步减少安全路径自身的组装成本。它让各 Rail 在同一个私有有序向量中借用互不重叠的区域，保留最后的发布复制，同时消除符合条件时的 packed 到 ordered 组装。这里的“直接”指直接写入私有有序暂存，仍然存在发布边界，不能理解成网络直接写入调用方的最终对象。

交付由可运行实现、确定性测试、真实 Verbs/RXE 收据和受控性能实验共同组成。2026 年 10 月 9 日的五组对照测量软件路径开销，10 月 12 日的版本切换实验验证身份复查拒绝分支，10 月 16 日的冻结验收与新演示覆盖 Python/C ABI 的生命周期恢复。不同日期的数据回答不同问题，正文不把它们拼接成一次统一的新实验。

本报告的贡献定位是版本化对象的读取与内存所有权契约，以及保持该契约时的暂存开销优化。上游 ContextStore 已有服务、协议、条带、SGE 与分层存储基础。已有证据支持真实软件 RoCE 执行和有限条

件下的软件开销收益；物理双 HCA 聚合、端到端推理收益与完整形式化正确性仍有各自的验证任务。

源码与证据：[S01] [S02] [S03]

02 应用场景与需求：对象、版本和内存有不同的完成时刻

ContextStore 的 Python 部分对接 vLLM/Dynamo 的 KVConnector，并区分 Scheduler 与 Worker。Scheduler 查询索引、构建跨侧元数据；真正的存储 I/O 由 Worker 执行。RailFuse 的读取引擎位于 Rust 客户端的数据路径，不要求把大模型或 Agent 放进延迟敏感的传输循环。条带选择、边界检查与资源释放需要确定且可测试的规则，现有实现采用显式状态和条件分支。

一次业务读取包含多种“完成”。某条 Rail 完成，表示它报告的字节和条带数达到计划；对象完整，表示全部必需条带已经到齐且现有校验通过；版本可接受，表示传输后的对象身份和实际 Placement 仍与初查相同；内存可回收，表示对应 QP/MR 的生命周期已结束。任何一个较早发生的事件，都不能替代后面的条件。例如 Rail 0 已经完成，并不允许发布缺少 Rail 1 的对象。

典型成功流程从 LookupObject 开始。客户端拿到对象身份与条带列表，验证布局后预留资源，启动各轨任务，将数据接收到私有区域。所有已启动 Worker 退出后，读取引擎检查完整性与可用的条带校验值。高层 SDK 再查一次对象身份，身份一致且取消未赢得发布门时，才把完整内容复制到调用方。调用方看到的是一个成功结果及其有效长度。

失败流程则强调外部可观察结果。布局无效和预算不足在传输前拒绝；路径错误、短读、线程异常、校验不匹配或版本变化使整次 Rust 读取失败；已经启动的任务仍需结束和清理。没有对失败 Rail 做同一次请求内的透明重试，因为重试会引入新的时间窗口、版本与资源组合。上层可以在错误返回并确认资源已收敛后，重新发起一次完整读取。

业务价值需要逐层验证。更少的内存搬运可能减少 Worker 的 CPU 与常驻内存压力，多路径能力可能在合适硬件上分担网络负载；这些是可测机制与后续假设。当前报告没有把对象微基准的比例直接换算成首 token 时延、吞吐或 GPU 利用率。真实推理验证还需要固定模型、前缀命中率、对象尺寸分布、并发和存储供给，再测量完整链路。

源码与证据：[S01] [S04] [S05]

03 系统边界与上游归属：增量落在客户端读取契约

仓库包含相互解耦的 Python Connector、Rust KVService、Rust client SDK、供 ctypes 使用的 rdma-ffi，以及 NIXL 插件。RailFuse 的多轨对象读取位于 client-rs 的 rail_read 模块，由 KvClient 提供带前后元数据查询的公共入口。Python 的 RdmaClient 通过独立 C ABI 加载动态库；它不直接调用私有 Rust RailReader，也没有自动继承多轨原子发布能力。

上游提供了对象存储服务、gRPC 元数据访问、条带布局、描述符读取、tag-12 条带子集和 tag-15 SGE 协议，以及 RDMA WRITE 数据路径。作品沿用这些设施，把既有 Placement 中的“谁拥有哪个条带”与客户端的“通过哪条本地设备和远端 listener 访问它”分开。网络路径的变化不要求重新编码 KV 数据，也不要求重写磁盘上的条带。

增量机制主要包括 RailRoute 与能力发现、整条带调度、共享 Reader 的资源准入、私有暂存与发布门、直接有序暂存，以及围绕异常完成的回收控制。服务器原有数据路径也必须满足源内存保活：已经发出的 WRITE 完成状态不明确时，不能先释放源 slab 或回退缓冲区，再继续在同一连接处理其他请求。客户端的目的内存与服务端的源内存是同一 WRITE 的两端义务。

协议兼容采用附加可选字段。Placement 可以携带某个已拥有条带节点的 fabric/listener 能力，旧 protobuf 客户端会忽略该字段。旧服务器没有这些能力时，客户端仍可使用显式 RailRoute。能力发现本身没有推断 GID、主动探测可达性或协商所有协议版本；启用 tag-15 的客户端仍需与具有对应处理逻辑的服务端配套。

开源归属与交付状态分别记录。独立实现与研究分支属于可审查的源码增量，实验工具和竞赛材料也有各自的日期与哈希。提供链接、发布分支或准备附件不等于维护者已合并，更不等于主办方已收件。本次报告更新只扩充解释和证据组织，不修改核心运行时源码；核心安全代码以 39ce2d5 开头的完整提交标识追溯。

实现层次与责任

层次	现有职责	本报告关注点
Python Connector	索引、编解码、Scheduler/Worker 分工与分层访问	应用集成位置；不把微基准等同于推理结果
Rust KVService	元数据、条带、服务端 WRITE、SGE 与源缓冲区	版本服务与不确定 WRITE 的源内存保活
Rust client-rs	gRPC 与原生 RDMA 客户端	规划、私有暂存、二次身份检查、预算与发布
rdma-ffi / Python	单连接外部缓冲区 GET/PUT	安全 ABI、错误前退休、CQ 排空与重连

源码与证据：[S01] [S02] [S06] [S07]

04 元数据契约：先证明条带描述自洽，再打开数据连接

ObjectDescriptor 记录 key、object_handle、object_generation、content_etag、layout_version、size、is_striped、stripe_count 与 chunk_size。handle 由服务端生成，客户端把它当作不透明身份，不解释内部文件路径。PlacementDescriptor 记录该对象的实际条带及归属，包括节点、RDMA 端点、存储句柄、对象内偏移、长度和可选校验值。身份信息回答“读哪个对象版本”，Placement 回答“具体字节在哪里”。

RailPlan::build 首先检查 key 一致、handle 非空、Generation 和 Layout Version 非零，拒绝空对象和不能转换为本机地址空间的尺寸。条带对象要求 chunk_size 非零，stripe_count 必须等于 size 对 chunk_size 向上取整的结果；条带数不能超出线协议可表示的范围，Placement 数量也必须相等。这些检查发生在建立传输任务之前，避免把无效元数据变成任意内存访问。

随后规划器按 stripe_index 构造有序表，拒绝重复、缺失和越界条带。第 i 个条带的对象偏移必须为 i×chunk_size，其长度必须等于 chunk_size 与剩余对象长度的较小者，RDMA 端点不得为空。这既覆盖普

通等长条带，也覆盖最后一条较短的尾块。每个条带因此只有一个对象区间；所有条带顺序拼接后应恰好覆盖完整对象。

完整性策略与版本策略分别检查。Placement 的条带校验值允许全部为空，以兼容早期对象；也允许每条都存在；只填一部分会被拒绝。若提供校验值，客户端按对象区间计算锁定实现的 xxh3-64 并逐条比较。若全部为空，客户端仍核验边界、字节和身份，但不能宣称检测所有静默字节损坏。服务端开启 verify_stripe_checksums 后，还要重写测试对象才会得到带校验的新 Placement。

传输结束后的身份比较覆盖上述描述符字段及完整 Placement，而不只比较 key 或 Generation。这样布局、条带归属、尺寸或内容标识变化都会导致拒绝。当前方法建立的是显式假设下的身份一致条件：相关覆盖应改变被检查的身份或校验值。它没有证明所有外部元数据修改、身份碰撞或未约束的 ABA 情况；这些需要服务端版本分配契约和更完整的并发分析。

源码与证据：[S05] [S06] [S08]

05 路径模型与发现：保持存储归属，显式选择网络

RailRoute 包含稳定 id、Placement 广告的 owning endpoint、真正拨号的 RdmaClientConfig、启用状态与正权重。一个对象可以通过同一节点的不同 listener 访问，但不能据此改变该节点拥有的条带。手工模式允许把同一个 advertised endpoint 映射到另一 fabric 上的 listener；这解释了演示中第二条 Rail 仍以 10.31.0.2:55153 作为归属标识，却连接 10.32.0.2:55154。

Reader 构造阶段检查路由唯一性。id、广告端点和 listener 不得为空，权重不能为零；相同 owning endpoint 下的 listener 组合、本地 device/port 组合都不能重复。最多允许 32 个已配置 route，超出时在打开 RDMA 连接前返回资源错误。该上限控制单次读取可能创建的线程和 QP 数量；它不是链路带宽自适应，也不表示实际会同时使用 32 轨。

发现模式把远端能力与本地信任配置结合。服务端只为拥有该对象条带的节点提供 RdmaRailEndpoint；本地 LocalRailPath 指定 fabric_id、device、port、gid_index 与 weight。客户端匹配 fabric 后构造 route，拒绝空身份、重复 fabric/listener、同一端点声称两个 owner，以及没有任何本地 fabric 可匹配的存储 owner。远端广告不能让客户端任意选择一个没有配置过的本地设备。 对未配置的额外 fabric 能力，代码可以跳过；关键要求是每个对象 owner 至少得到一条本地可用映射。

发现结果保留原始 capability 与 owner 集合。后续读取若发现监听器能力或存储归属改变，会明确要求重建 Reader，不会悄悄把旧计划套在新拓扑上。对缺少 capability 的旧服务器，发现方法返回可解释错误，调用方可主动选择手工模式。这种行为把升级决策留在配置层，便于审查实际使用的网络和旧新版本兼容边界。

可用性状态来自启用开关和失败冷却。set_enabled 可以停用或恢复 route，读取前使用当时可用路径重新规划；传输错误、短读或 Worker panic 会使相应路径进入冷却，描述符过期则不被错误标记为链路故障。sysfs 能提供 NUMA 和 PCI 标识时会记录它们，RXE 缺少这些物理信息时显示为空。当前权重由配置指定，没有自动测量链路容量或自动执行 NUMA 亲和策略。

源码与证据：[S05] [S07] [S09]

06 条带调度：按已分配字节平衡，故障后重建下一次计划

默认 packed 调度以整条带为单位。规划器按对象顺序遍历条带，只在 advertised endpoint 与条带 owning endpoint 相同的可用 route 之间选择。选择依据是该任务已经分配的字节数除以正权重；实现用交叉相乘比较，避免浮点计算。归属不符的 route 即使空闲也不能接收该条带，找不到合格 route 时整次规划失败。

权重描述相对容量提示。相同权重的两条路径通常交替获得大小相同的条带；更大权重的路径可承担更多字节。由于整条带不能任意拆分，条带数较少、尾条带较短或 owner 分布不均时，实际比例不一定等于权重比例。每个 StripeRead 同时保留对象偏移和该轨 packed 偏移，避免把网络接收顺序误当作完整对象顺序。

以 64 MiB、16 个 4 MiB 条带和两条等权路径为例，默认贪心计划可以让两轨各接收 8 个条带，总载荷各 32 MiB。若条带交错分配，各轨缓冲区内部仍紧凑存储自己的 8 个条带；最终组装依据 object_offset 放回完整对象。这种计划适用于更一般的存储归属，而不要求所有条带集中在同一个 owner。

路径停用影响下一次计划。如果第二轨在请求开始前已停用，第一轨仍覆盖全部必需 owner，读取可退化为一轨。如果第二轨在传输途中失败，已启动任务会结束和清理，但当前请求不把失败条带重新塞给第一轨继续拼接。重新发起读取时会重新查询身份、过滤冷却路径和核算资源。这样每次接受的对象都有一个明确的初始身份与完整执行区间。

可观察输出不仅记录配置，还记录实际计划和有效模式。plan_assignments 给出每轨 stripe_indices 与 payload 字节；mode_counts 区分一轨、packed-greedy、packed-contiguous 和 direct。配置 direct_staging=true 不代表每次都符合直接路径条件；审核必须查看实际计数与条带分配。这也是实验审计要求保留 effective mode，而不只保留启动参数的原因。

源码与证据：[S05] [S08] [S10]

07 直接有序暂存：以互斥区域代替完整对象组装

packed 路径的安全性来自私有接收和最终发布，但两轨交错接收后仍需把各自缓冲区复制成一个有序对象。直接有序暂存保留同样的应用可见边界，改变私有内存内部的组织方式：先分配一个大小恰为对象 size 的向量，再让每个 scoped Worker 借用一个互不重叠的可变切片，RDMA 直接写入它最终所属的有序位置。

优化是显式启用的。RailLimits 的 direct_staging 和 contiguous_schedule 默认都为 false。连续重分配要求至少两条实际选中路径、路径数不多于条带数、owner 非空，且所有条带的 node_id 与 rdma_endpoint 相同。多 owner 布局或不满足条件的请求保留原规划。当前直接路径针对条带对象的部分条带任务，不把普通未条带对象强行转换成多轨传输。

连续规划先给每条 Rail 至少一条条带，再按权重把剩余条带分配配额；余数按确定顺序补齐。各 Rail 得到连续的对象区间，任务按 route_index 排序。64 MiB 等权例中，r0 获得条带 0—7，r1 获得条带 8—15。每个区间内部的 packed_offset 与它借到的目标切片对应，组合后的切片恰好覆盖整个私有向量。

Rust 的 split_at_mut 为这些切片建立互不重叠的借用关系，scoped threads 约束借用不能逃出请求作用域。所有 Worker join 后才检查校验值；任一错误或取消都丢弃整份暂存，调用方哨兵保持原值。每个 Worker

结束前先销毁 client/QP，再解除其 MR；共享向量只在全部借用结束后才能继续验证和发布。一次 Rail 错误不能提前释放另一轨仍在 DMA 的向量。

同调度 packed-contiguous 对照只改变接收与组装策略，使用与 direct 相同的连续条带集合。它可以区分“连续调度本身”与“少一轮组装”的影响。直接模式仍需 dummy 区域、MR 注册/注销、校验、第二次 Lookup 和最终调用方复制；因此它减少软件内存搬运，却不构成全链路零复制。报告把观测收益称为直接暂存实现效果，避免把总时延差全部解释为纯 memcpy 时间。

源码与证据：[S05] [S08] [S11]

08 SGE 映射与服务端回退：数据位置不能随着快慢路径改变

RailFuse 复用现有 tag-15 条带子集 SGE GET。请求携带所需 stripe_indices 和目的段表，服务端读取被选中的条带并把数据写到对应目标地址。客户端既要证明拥有条带的地址有效，也要让完整段表中的未选条带具有安全映射。否则服务端快慢路径对段表处理的细微差异，就可能把对象偏移错误映射到较小的 per-Rail 缓冲区。

packed 模式给每条 Rail 分配 packed_len 加 dummy_len 的接收区域。属于该 Rail 的条带指向紧凑 packed 区域，未归它的条带指向尾部 dummy。若该任务已包含全部条带，dummy 可以为零；否则 dummy 至少可容纳一个条带，实际取 chunk_size 与对象大小的较小者。构建段表时检查地址加法、注册边界和长度，不允许 SGE 越过 MR。

直接模式分别注册目标切片与独立 dummy。目标切片的长度严格等于该任务的有效载荷，所选条带在切片内依次紧凑排列；其他条带引用 dummy MR 的地址和 rkey。实现还检查每个 owned stripe 都完成映射，尾块长度不能超过对应注册区域。dummy 是协议映射所需的安全占位，不计入对象载荷吞吐，但必须计入暂存与注册资源预算。

服务端来源可能是注册 slab，也可能因为 slab 容量不足使用注册缓冲区回退。回退仍必须遵守相同 SGE 目的映射，不能退回“对象偏移直接加到某个紧凑基址”的错误规则。WRITE 已发布后若完成状态不明确，服务端还要保留源数据和 MR，直到相应 QP 被销毁；在同一 QP/CQ 上直接重试或继续复用源缓冲区会破坏这一生命周期前提。

历史 64 MiB 与 256 MiB 实验正好体现来源条件的重要性。服务端 RDMA slab 覆盖设置为 128 MiB，64 MiB 请求处于相应正常路径，256 MiB 超过 slab 并记录 registered-buffer fallback。每个尺寸内的五组对照使用同对象同来源条件，但不能把两个尺寸间的差异解释成单纯的对象尺寸效应。保留回退日志可以解释结果，也能验证优化没有只在理想 slab 命中时生效。

源码与证据：[S05] [S09] [S11] [S12]

09 资源预算：请求结束与暂存释放分两次结算

多轨会同时消耗线程、连接、锁页内存与目的缓冲区。RailReader 维护共享 BudgetState，以互斥锁一次核算准入，覆盖 active_reads、staging_bytes、registered_bytes、inflight_bytes，以及每轨的 active_reads 和 inflight_bytes。预算属于一个 Reader 实例，多个调用并发共享它；独立构造的多个 Reader 没有自动共享全进程总额，部署层仍需设置外层限额。

默认值为最多 8 次活动对象读取、每轨最多 8 个活动任务，暂存/注册/在途载荷总额各 4 GiB，每轨在途载荷 2 GiB，控制 I/O 超时 30 秒、失败冷却 10 秒。Linux 上若进程 soft RLIMIT_MEMLOCK 有限，max_registered_bytes 还会收紧到该软限制的 80%。这些是准入预约与头寸，不是硬件测得的实际 pin 页计数，也不代表允许程序无限接近系统资源耗尽。

令对象载荷为 S，各轨有效载荷为 P_r，占位为 D_r。注册预约为 R= Σ (P_r+D_r)，对象在途为 S，每轨在途为 P_r。packed 的暂存预约为 S+R；direct 的暂存预约为 S+ Σ D_r，因为 direct 的有效载荷区域就是那份有序向量。这解释了两者可以具有相同注册量和在途量，却有不同的暂存开销。单轨 handoff 的保守预算仍按现有 reserve 逻辑预留，不能把预约值直接当成精确分配峰值。

64 MiB、两轨、4 MiB 条带例中，各轨 P_r=32 MiB、D_r=4 MiB，因此注册预约合计 72 MiB、有效在途合计 64 MiB。packed 预留暂存 136 MiB，direct 预留 72 MiB；这些值不包含进程其余对象、分配器和运行库开销，因此 RSS 不应与它们机械相等。预算超额时返回 ResourceExhausted，不先启动部分 Rail 再等待其他 Rail 凑齐资源。

BudgetGuard 在所有传输结束后调用 finish_transfer，释放 transient staging、注册和在途预约，并扣减各轨活动任务；但完成对象的最终暂存与 active_read 名额继续保留在 StagedRailRead 中。高层第二次 Lookup、版本比较和发布期间仍占有它们；发布或丢弃后才通过 Drop 释放剩余预约。这避免“大对象已经传完但尚未发布”期间预算被提前归还，又接纳过量新请求。

64 MiB 双轨例的预算

项目	packed-contiguous	direct
有效载荷	64 MiB	64 MiB
每轨载荷 / dummy	32 / 4 MiB	32 / 4 MiB
注册长度预约合计	72 MiB	72 MiB
有效在途载荷合计	64 MiB	64 MiB
暂存预约	136 MiB	72 MiB
传输结束后保留	64 MiB 最终暂存 + 活动读取名额	64 MiB 最终暂存 + 活动读取名额

源码与证据：[S05] [S08]

10 发布契约：完整性、二次身份查询和取消共用一道门

高层 KvClient::read_multi_rail_into 接受 Arc<RailReader>、namespace、object_key、调用方可变缓冲区和可选 RailCancel。初次查询未找到对象时返回 Ok(None)；缓冲区过小在传输前报错。同步 RDMA 读取放入 spawn_blocking，异步运行时不直接执行阻塞 Verbs 调用。成功暂存返回后，SDK 再执行 LookupObject，并在发布前比较初查与复查结果。

比较包含 key、handle、Generation、ETag、Layout Version、尺寸、条带标志、条带数量、chunk_size，以及完整 Placement。复查对象消失也作为 VersionChanged 拒绝。payload 长度必须恰好等于对象 size，destination 必须足够大；成功只复制 destination 的前 size 字节。这样的契约允许调用方预分配较大缓冲区，同时避免把未定义尾部误认为对象有效内容。

取消由 RailCancel 共享原子状态和 publish_gate 互斥锁。cancel 在同一门内设置取消标志；publish_if_live 进入门后检查标志，再完成最终复制。因此取消先赢得门时，读取返回 Cancelled 并保持调用方内容；发布先开始时，取消不能撤回已经完成的复制。取消没有把在途 Worker 强行中止，线程仍需 join 和结束 DMA 生命周期，错误返回可能晚于取消请求。

三层 API 的语义不同。read_staged 返回已完成并核验现有校验值的私有对象，调用方仍需执行版本复查；低层 read_into 可以复制已完成结果，但其文档明确不在传输后重新查询对象；只有高层 read_multi_rail_into 将前后查询、暂存、完整性与发布门串成完整流程。使用示例应优先展示高层入口，不能把低层便捷方法描写成同样的并发覆盖保证。

日志也需服从结果语义。高层接受或版本拒绝诊断使用不改变返回值的写入方式：stderr 关闭不能把已经发布的数据重新报告为失败。CS_RAIL_TIMING 可以记录 accepted identity、前后 Generation 与阶段耗时，便于审计；它仍属于可选诊断，不是新的同步机制，也没有给元数据服务增加事务隔离。线性化点和所有并发重写交错的完整证明超出当前证据。

源码与证据：[S04] [S05] [S08]

11 Rust DMA 生命周期：先退休连接，再解除注册

RDMA GET 的数据方向容易造成误解：客户端发起读取控制请求，数据由服务端 RDMA WRITE 写入客户端已注册区域。TCP 读取失败只说明控制面没有收到预期结果，不说明服务端没有发出 WRITE。因此错误处理必须处理数据面仍可能持有目的地址的情况，目的向量也不能随着 Result::Err 的产生立刻释放。

每条 Rail 的 VerbsTransport 创建独立 RdmaClient，并注册自己的 packed 区域或 direct 切片。传输返回前显式 drop(client)，随后 drop(registered)。RdmaClient 的 Drop 先销毁 QP；控制错误也会提前退休 QP 并关闭连接，阻止后续请求复用旧控制流。RegisteredBuffer 带有对目标切片的借用，并保留 PD/CQ/device 资源，避免 MR 尚存时其依赖已经消失。

服务端也承担对称责任。WRITE 可能仍读取其源 slab extent、缓存 pin 或临时注册字节。完成不明确时，这些资源需要随 RcQp 保活，直到 QP 销毁。服务端不能在未知 WRITE 后返回到同一 CQ 继续处理请求，也不能让旧 CQE 充当后续写的完成凭证。complete-object GET、条带子集与 fallback 的路径都需要遵守同一个源内存规则。

清理函数本身也可能失败。当前 Rust 客户端对 ibv_destroy_qp 或 ibv_dereg_mr 的失败采用 fail-stop：用低层输出记录尽力诊断，然后终止进程，避免异常展开把仍可能暴露给 DMA 的内存回收。这样的代价是牺牲进程可用性，换取不在所有权不明时继续执行。测试用返回值注入覆盖了该分支，但尚未重现真实 provider 硬件清理失败。

Worker 的 panic 会被捕获并归一为 WorkerPanic，已启动线程仍全部 join，活动计数由守卫归还；调用方缓冲区依旧未发布。真正清理失败使用 abort，不能被普通 catch_unwind 吞掉后继续复用内存。整个安全论证依赖“成功销毁 QP 后不会再通过它发生未来 WRITE”等 provider 语义；真实 RXE 收据检验若干关键交错，有限状态模型表达这些前提，两者都不能替代所有 provider 的实现证明。

有限状态模型中六个守卫的作用（抽象反例，不是 provider 实测）

必要守卫	去掉后可到达的反例	对应实现责任
目标复用前退休 QP	超时 → 注销 MR → 复用目标 → 晚到 WRITE 污染	client/QP 先于 MR 与目标释放
私有暂存保留到发布	两轨结束 → 检查 → 复用暂存 → 发布已失效字节	StagedRailRead 保有完整向量与预算
源释放前退休 QP	发出 WRITE → 超时 → 源缓冲区提前释放	服务端 RcQp 保有 slab/pin 或 MR/Bytes
完整覆盖后发布	短条带有完成事件，却被视为完整对象	规划覆盖、回报字节和 chunks 核对
取消与发布共用门	检查通过 → cancel 返回 → 仍然发布	RailCancel::publish_if_live
二次身份必须匹配	对象 A→B → 忽略复查差异 → 发布	commit_if_unchanged 与完整 Placement 比较

源码与证据：[S05] [S09] [S12] [S13] [S24]

12 Python 与 C ABI：明确单连接和外部内存责任

Python RdmaClient 是单连接封装，提供内置缓冲区 GET、外部注册区 get_into、descriptor GET 和 PUT。ctypes 会进入原生库，Python 层用每句柄 Lock 串行化连接、传输、注册、注销和关闭。C 调用者没有该自动锁，必须自己串行化同一个 handle 的全部操作，包括 free。这个接口服务于受控内存复用，没有多轨对象私有暂存与高层原子发布。

初始化先验证 ABI 能力。动态库必须导出 cs_rdma_client_lifecycle_version，且返回至少 1；缺失或版本过低在 native allocation 之前拒绝。显式 timeout 还要求原生 deadline 支持；当前构建默认控制超时为 30 秒，范围是 DNS 解析之后的 TCP 操作，不限制 DNS 本身。端口、GID、字节长度、指针范围、字符串 NUL 与整数溢出都在进入危险边界前检查。

FFI 的 settle_on_error 统一包裹可能失败的连接和传输。失败时先关闭控制流、销毁 QP，再排空该句柄共享 CQ 中的旧完成；Python 随后把 connected 标记清除，下一次传输前需要 connect。之所以还要排空 CQ，是因为重连可复用同一个 CQ，旧 PUT 的成功 WC 若残留，可能被新 PUT 错认成自己的完成。QP 编号可能被 provider 重用，不能用号码变化代替生命周期判断。

外部注册不会把内存所有权交给库。调用方必须保证分配持续存在、注册期间不释放、不被并发传输和业务写入冲突；借出的 memoryview 不能跨后续传输或 close 保留。内置 get 加 buffer_view 的消费也需由调用方作为一个整体串行化，单次方法锁不会保护两次方法之间的业务竞态。close 先从 Python 对象摘除 handle 再 free，避免中断造成悬空句柄被重复释放。

失败语义需要准确表达：get_into 失败时可能已经收到部分数据，这块外部缓冲区不能被业务当作完整对象消费；修复版保证错误返回前完成既定退休流程，使后续复用和重连遵守安全边界。它没有承诺失败时字节不变。若 Python 需要与 Rust 多轨一致的原子发布，后续必须设计专门的公共 C ABI、持有私有对象与预算，并公开可审计的发布契约。

源码与证据：[S14] [S15] [S16]

13 故障分类与恢复：错误名称要指导下一次动作

错误类型把前置条件、数据完整性、版本冲突和基础设施故障分开。InvalidPlacement 表示描述或路径配置不自洽，ResourceExhausted 表示预算拒绝，BufferTooSmall 给出需要与现有容量。这三类应先纠正输入或降低并发，重复发送同样请求不会解决根因；它们也不应被统计成网络失效。

Transport、Incomplete 和 WorkerPanic 会使相关 Rail 进入冷却。冷却期间的新请求重新选择其他可用路径，若其覆盖所有 owner 可以用更少 Rails 完成；若不能覆盖则返回明确错误。StaleDescriptor 与 VersionChanged 表示对象身份不再适用，并不直接证明网络故障，因此不把健康路径下线。Checksum 表示内容与已公布条带摘要不符，应保留对象、版本和条带号定位来源。

Rust 在已经开始的传输中不承诺快速取消。Cancelled 的业务意义是取消赢得最终发布门，调用方不接收部分数据；内存和 Worker 的结束仍然受实际连接和 provider 清理约束。超时也有层次：控制 I/O deadline 与服务端 CQ deadline 不相同，当前原生 PUT 完成轮询还具有独立的 30 秒行为。调低控制超时并不能证明所有阶段都会在同样时限内返回。

恢复流程先确认旧请求已经收敛，再发起新读取或重连。Rust 上层重新执行 LookupObject，不能沿用已经判定失效的描述符；Python FFI 传输错误后显式 connect，同一外部 MR 可以在遵守生命周期前提下复用。测试要求核验恢复内容而不只核验 connect 返回成功，因为新的握手并不能排除旧完成污染后续 PUT。

运行诊断应同时保留路径 id、设备与 listener、对象身份、错误类型、每轨字节与当前预约。结束态 inflight 和 registered reservation 归零说明账本收敛，不等于硬件内部所有细节已被观察。对于 provider 清理失败，当前策略是 fail-stop，恢复交给进程或服务重启边界；自动在原句柄上忽略错误继续运行会违反设计前提。

错误与处理建议

类别	调用方可见结果	恢复方向
布局 / 容量 / 预算	Rust 发布前拒绝	修正描述、缓冲区或并发资源配置
传输 / 短读 / 线程异常	整次 Rust 读取失败，相关路径冷却	等待收敛后完整重读，检查链路与原始日志
描述符过期 / 版本变化	不发布该次 Rust 暂存	重新查询身份后重读
条带校验不匹配	不发布并给出 stripe	核查源数据、摘要与版本，保留异常证据
FFI 传输错误	外部区可能部分写入，句柄需重连	停止消费失败内容，重连后完整 PUT/GET 验证
QP/MR 清理失败	进程 fail-stop	保留诊断，使用进程级恢复边界

源码与证据：[S05] [S09] [S14] [S15]

14 验证体系：每层证据都对应一个可观察契约

最底层的 Mock 测试验证规划与发布规则：条带缺失、重复、越界、错误 key、部分校验、加权分配、单轨退化、冷却恢复、预算拒绝、取消和线程 panic。direct 的专门用例核对连续互斥区间、独立 dummy MR、部分第二轨失败不发布、checksum 失败保持哨兵，以及 packed-contiguous 与 direct 计划完全一致。它们适合确定性边界检查，但没有真实 WR。

资源测试跨并发请求检查每轨在途上限和任务上限；StagedRailRead 生命周期测试确认传输结束后仍保留最终暂存与 active_read 预约。单轨 handoff 测试检查接收向量的指针身份，避免以“结果正确”掩盖不必要的完整对象复制。取消与发布测试验证统一门的先后关系，而不仅检查某次调用最终返回了错误。

原生生命周期探针使用确切事件序列检查初始化失败清理、传输失败退休、释放与 fail-stop。2026 年 10 月 16 日冻结材料记录 13/13 native cases、6/6 解释器优化与收据校验，以及 Python 165 passed、11 skipped；完整 build、server tests 和既定格式/静态检查的退出文件均为零。skip 仍保留 skip，不能从合计中推断被跳过能力已运行。

真实 RXE 冻结矩阵有 12 份接受收据：两条链路各一个旧库 exact-corruption 负对照；候选 GET 覆盖两链路、1/8 MiB、畸形回复与 100 ms 控制超时，共 8 例；另两例是各链路一次受控 CQ 延迟 PUT，随后各做 3 轮同句柄同注册区 PUT/GET。每份收据绑定库、模块、探针、环境及时间序列。

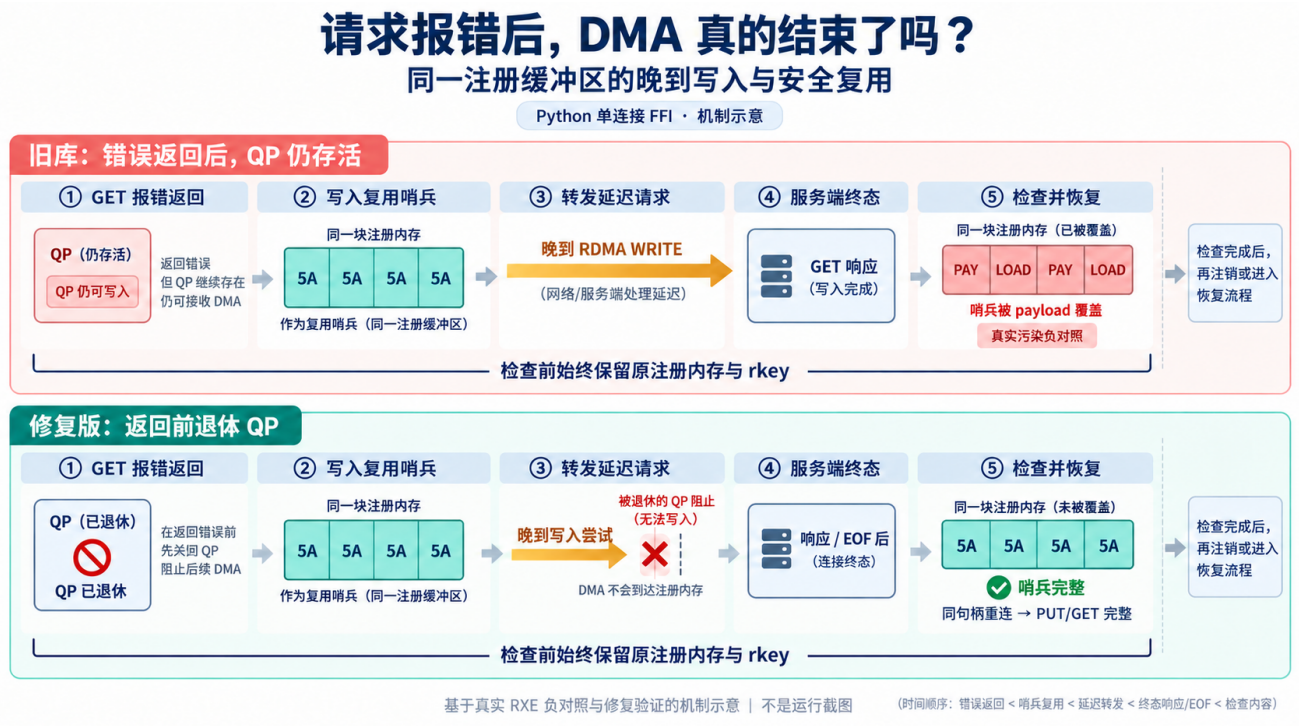
最后是独立审计与演示。冻结文件有 source manifest 和源码审查记录，新 V2 演示另存 launch/completed 哈希、命令、stdout、退出码与事件流。本次报告复用已保存证据，没有重新运行这些核心测试或改写历史结果。主流程退出为零表示验收驱动正确识别了预期结果；旧库故意损坏的探针仍必须退出 1，不能为了得到“全绿”抹掉负对照。

源码与证据：[S08] [S16] [S17] [S18]

15 晚到 WRITE 负对照：同一注册区必须留到终态观察之后

晚到写入的验证必须证明错误返回之后原地址仍可能被访问，而不只是制造一个超时。探针先验证正常 classic/external GET 内容，再由 TCP relay 截住一个真实 GET 请求。畸形回复让客户端先返回错误，随后将同一外部注册区填成哨兵；relay 延迟后把原请求转发给真实服务端，最后等待服务端完整回复或 EOF，再检查原注册区。

关键是保持同一块 allocation、同一个 rkey 和注册状态。若探针提前注销 MR、换新内存或在服务端完成前就检查哨兵，坏实现也可能看起来安全。当前探针拒绝优化解释器，避免 assert 被消除；还拒绝把普通等待超时或截断响应当成服务端终态。时间戳记录 relay 收到、API 错误返回、哨兵复用、延迟转发、服务端终态和最终检查的先后顺序。



晚到 WRITE 对照示意。旧库在 API 错误后仍把旧 payload 写入复用区；候选先退休 QP，再允许复用。图解释时序，实际判据见 baseline.json 与 fixed.json。

V2 旧库负对照在 1 MiB 负载上观察到 exact_payload_corruption=true、reused_sentinel_intact=false，哨兵最终 SHA-256 与预期对象 payload SHA-256 相同。它证明测试中确实发生了目标内存被真实旧请求数据覆盖，排除了“随机错误或非零退出”造成的模糊判据。旧库只有在隔离 negative probe 的显式 unsafe-baseline 模式才会绕过普通包装器的 ABI 拒绝。

候选库同样在错误返回后复用仍注册的地址，但它已经退休 QP。此次收据等到延迟服务端进入 EOF 才检查，结果 reused_sentinel_intact=true，exact_payload_corruption=false；随后在同一个 native handle 上重连并核验 PUT/GET 完整内容。错误发生在 relay 转发之前，检查发生在服务端终态之后，这两个时序条件共同构成该用例的安全证据。

冻结矩阵对两链路、不同载荷和错误方式做了更宽覆盖；V2 新演示只重演其中一条链路的代表案例。两者都在两台 KVM guest 的 RXE provider 上执行真实 Verbs，relay 改变控制面时序，未伪造 provider 完成。

它们覆盖的是明确构造的一组交错，不能由单次无污染推导任意硬件、任意拥塞和所有晚到 WR 都不会出问题。

源码与证据：[S16] [S17] [S18]

16 PUT 与旧 CQE：重连成功必须包含真实完成归属

PUT 的风险与 GET 不同。客户端 WRITE 将外部注册源数据推送到服务端，完成通知从 CQ 返回。即使旧 QP 已销毁，如果它留下的成功 WC 仍在句柄复用的 CQ 中，新连接上的下一次 PUT 也可能把旧完成当作新写已结束。仅检查 QP 销毁或重新建立连接，不足以证明源缓冲区和提交顺序安全。

当前 FFI 的 retire_connection 在销毁 QP 后循环 ibv_poll_cq，消费旧/flush completions，直到返回零。CQ 返回负值无法确认收敛时同样 fail-stop。新 connect 先退休原连接，再建立新 QP；普通传输错误经过 settle_on_error 后才向调用者返回。这样共享 CQ 的历史完成在 API 边界前被清理，后续操作不会借用旧凭证。

验证使用 test-only CQ observer：QP 活动期间故意把 CQ polling 结果压成零，等到 QP 销毁后恢复调用真实 provider，并记录真实 WC 的 qp、wr 和 status。它没有合成 WC，却主动制造完成不可见条件，因此应称为受控 poll suppression。完成轮询约 30 秒后触发错误，旧 QP 的真实 WC 必须在 API 错误返回前被 observer 看到和消费。

V2 的 put-cq.json 记录 old_qp_real_completion_drained_before_error_return=true、same_registered_buffer_reused=true，并完成 3 轮 reconnect PUT/GET 全内容验证。保留事件显示 QP 销毁之后真实 completion 被揭示；随后复用同一 native handle 和同一注册区。冻结矩阵在两条 RXE 链路分别执行了该用例，每链路均有三轮恢复，属于更完整的验收。

测试不要求新旧 QPN 不相同。RXE 可以立即重用销毁后的号码，号码相同不表示旧 QP 仍存活；号码不同也不能证明 CQ 没有旧 WC。有效判据是完成顺序、错误返回位置和恢复数据。当前结果没有证明无注入的自然 PUT timeout 场景，也没有测量硬件 CQ 在真实链路故障中的全部 flush 行为；这些被列为后续 provider 验证任务。

源码与证据：[S14] [S16] [S18]

17 公平基线与负结果：更快的一轨改变了多轨结论

最初的软件 RoCE 一轨/两轨对照出现过约 1.038—1.101 倍的表观两轨收益。随后发现一轨路径在已经收到完整有序对象后，仍额外分配并复制一次完整对象。这样的基线会把避免的软件开销误记为多路径收益。修正后，只要唯一 Rail 的接收区大小精确、覆盖完整、packed_offset 等于 object_offset，便把已静止的私有向量移交 StagedRailRead，保留校验和最终发布门。

物理单 HCA 的代码版本消融使用相同 64 MiB/16 条带对象、相同 server/config 与 mlx5_1 路径，10 个独立进程对采用 AB/BA 交替顺序。优化后一轨的配对旧/新延迟比几何均值为 1.426，95% block bootstrap 区间为 1.342—1.534，10/10 区块支持优化。CPU 与 RSS 同时下降，支持冗余复制确实有实际成本；这是一轨应用代码优化，不能称为网卡聚合。

使用优化后同一客户端二进制重做 RXE 矩阵，64 MiB、并发 1 的 two/one 吞吐比为 0.904，95% 区间 0.874—0.934，10 个配对区块全部偏向一轨。256 MiB/并发 1、64 MiB/并发 4、256 MiB/并发 2 的区间都跨 1，CPU 和 RSS 均更偏向一轨。应保留这些结果，因为它们说明“增加 Rails 就更快”不适用于当前软件栈。

256 MiB 并发 2 的两组都使用 unlimited memlock，其余单元使用 guest 默认限制；同一比较单元内政策一致。不同修订的矩阵运行于不同 VM session，不能把前后比值直接当成随机化代码因果估计。物理 AB/BA 消融才是冗余复制修正的受控证据。历史文档中“直接多轨尚未实现”的句子属于当时状态，后续 10 月 9 日实现与实验已单独更新，时间不可混用。

这一负结果推动了真正可检验的下一步：保持强一轨基线，用相同条带调度比较 packed 与 direct，观察减少组装能否降低安全暂存开销。它收窄了主张，也让性能优化有明确成本来源。竞赛展示保留该过程，可以解释为何作品重点是完整版本、安全所有权和更少软件搬运，而不是把两个逻辑路径包装成已实现双倍物理带宽。

修正基线后的历史 RXE 对照

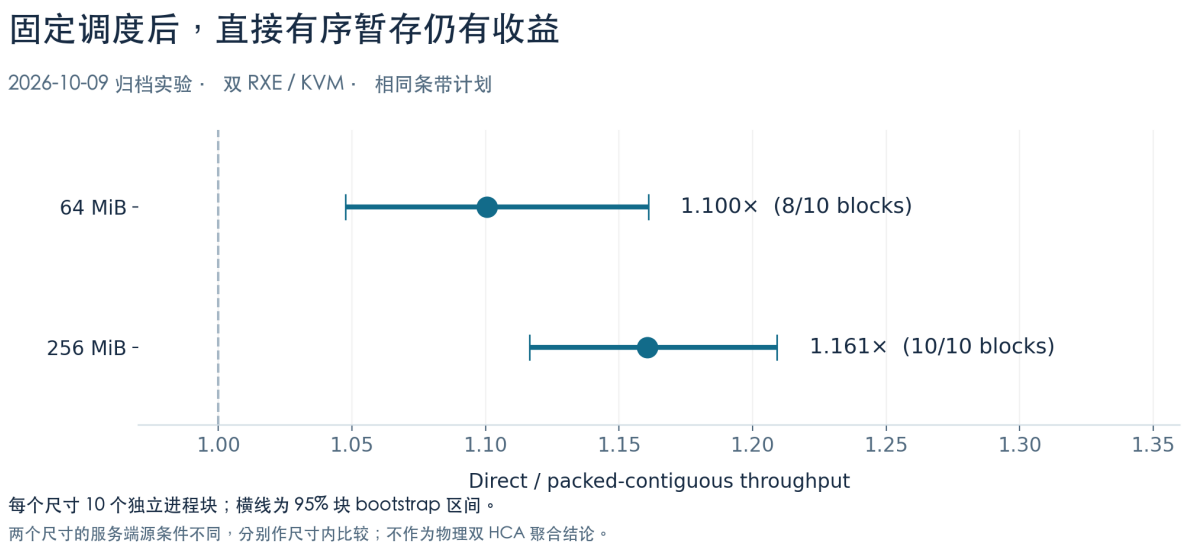
对象 / 并发	two/one 几何均值	95% 区间	支持两轨区块
64 MiB / 1	0.904×	0.874—0.934	0/10
256 MiB / 1	0.958×	0.908—1.013	4/10
64 MiB / 4	0.991×	0.959—1.022	5/10
256 MiB / 2	1.037×	0.995—1.077	8/10

源码与证据：[S19] [S11]

18 受控性能实验：固定计划，测量直接暂存的净效果

2026 年 10 月 9 日实验设置五组：rxe_c0 一轨、rxe_c1 一轨、两轨 packed-greedy、两轨 packed-contiguous、两轨 direct-contiguous。预先指定的主要比较为 direct/packed-contiguous，两者条带集合完全相同。该比较估计直接暂存实现的总效果，其中包括内存分配、目标/dummy MR 注册和销毁差异；不是把总耗时差孤立归因于单条 memcpy。

客户端与服务端是同一宿主机上的两台 4 vCPU、4 GiB Ubuntu KVM guest，两组 virtio 网络经独立 bridge 形成 RXE 路径，磁盘目录共享服务端虚拟盘。每个尺寸执行 10 个独立启动的 process block，区块内五组均运行；采用五种循环顺序及逆序，使每组在每个位置出现两次。每进程预热 1 次、测量 5 次完整对象读取，避免把同进程的 50 次读错误当成 50 个独立样本。



由 10 月 9 日原始审计 JSON 生成的性能图。主要比较固定调度；结果属于软件 RXE 与该 VM 会话。

审计重新计算每进程中位数，核对完整 hash、前后 descriptor/Placement 指纹、每轨条带索引、精确字节覆盖、实际模式与结束态资源。比值按配对区块的吞吐对数统计，再报告几何均值；95% percentile bootstrap 对 10 个区块进行 20,000 次有种子重采样。区间描述当前 VM-host 设置的区块变异，次要比较没有做多重比较校正。

主要结果为 64 MiB 的 1.100432 倍，95% 区间约 1.0475—1.1609，8/10 区块超过 1；256 MiB 为 1.160570 倍，区间约 1.1166—1.2092，10/10 超过 1。相应 CPU ms/read 从 123.9 降为 76.5、433.6 降为 259.4，峰值 RSS 从 206.3 降为 142.5 MiB、782.5 降为 526.6 MiB。CPU 包括预热和进程初始化摊销，时延比仅用测量读的中位数，两者统计口径不同。

直接与同计划 packed 的每轨峰值注册量之和仍分别为 72/264 MiB，在途载荷为 64/256 MiB。收益不是靠减少实际对象字节或取消预算取得；个别峰值求和也不等于同步全局峰值。另一正常路径 session 保留主要收益，却使 64 MiB direct/rxe_c0 区间跨 1，因此“一轨普遍更慢”缺乏跨 session 稳健证据。两个尺寸的源 slab/fallback 条件不同，所有推断限定在尺寸内部。

固定条带计划的主要比较

对象	Direct / packed-contiguous	95% block bootstrap	超过 1 的区块
64 MiB	1.100432×	约 1.0475—1.1609×	8/10
256 MiB	1.160570×	约 1.1166—1.2092×	10/10

同实验的资源口径

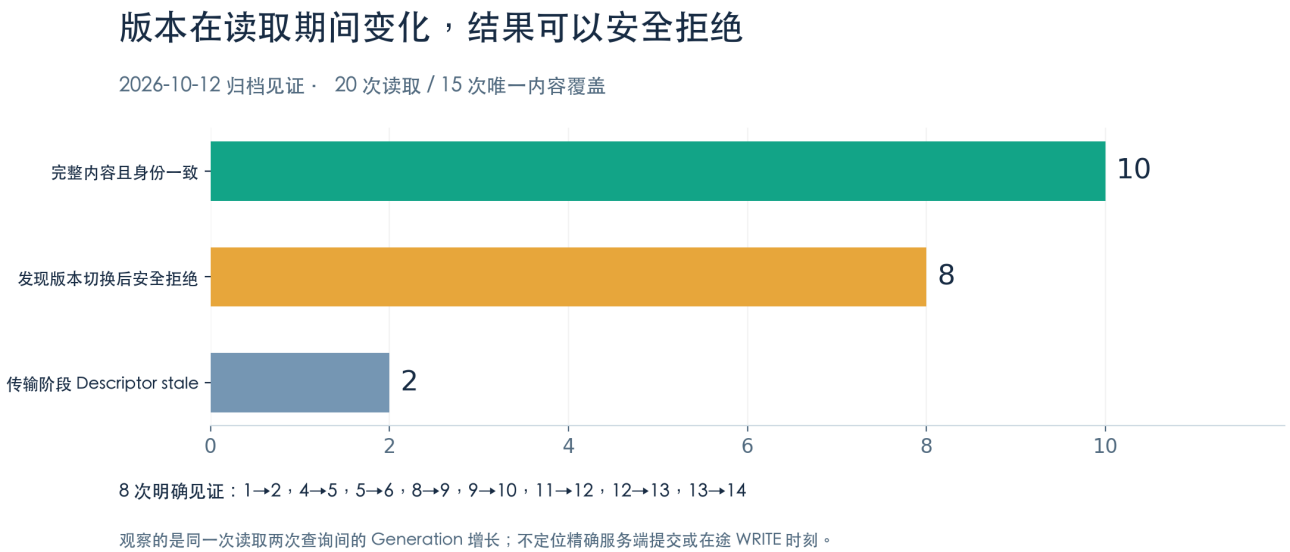
对象 / 模式	CPU ms/read	峰值 RSS MiB	各轨峰值注册 / 在途 MiB 之和
64 / packed-contiguous	123.9	206.3	72 / 64
64 / direct	76.5	142.5	72 / 64
256 / packed-contiguous	433.6	782.5	264 / 256
256 / direct	259.4	526.6	264 / 256

源码与证据：[S11] [S20] [S21]

19 Generation 切换见证：复查版本确实在真实执行中阻止发布

并发写实验首先要区分三个不同事件：读取方法与客户端 PUT 方法区间相交，读方法前后查到不同元数据版本，以及服务端提交与某个 RDMA WRITE 在途区间相交。前者只能证明客户端调用时间重叠；后者需要服务端关联追踪。10 月 12 日实验使用同一次读的两次 LookupObject，获得中间至少发生元数据替换的更强见证，同时保留无法定位精确提交时间的边界。

该轮在双 RXE 上运行一个顺序读客户端和一个唯一内容覆盖写客户端，不删除对象。共 20 次 8 MiB 双轨读、15 次唯一内容覆盖。写端在计时 PUT 前准备好 payload；读端打开 CS_RAIL_TIMING，版本拒绝时记录 initial_generation 和 current_generation。每份唯一内容可与 SDK 接受身份比较，避免只用重复 A/B 内容掩盖不同 Generation。



10 月 12 日独立版本切换见证：10 次接受、8 次前后 Generation 增长拒绝、2 次 stale 拒绝。历史数据与 10 月 16 日演示分别标日期。

结果中 10 次读取返回完整数据，内容编码的 Generation 与 SDK 接受身份相同；另 10 次拒绝并保持调用方哨兵。拒绝中 8 次观察到严格递增的前后 Generation，分别为 1→2、4→5、5→6、8→9、9→10、11→12、12→13、13→14；另外 2 次在传输阶段报告 descriptor stale，没有形成完整前后查询见证。不存在对象缺失

，两个 Rails 均传输了非零字节。

在一个 writer、无 delete、服务端生成递增 Generation 且 Redis 原子替换元数据的这些条件下，G_before<G_after 说明至少一次元数据替换发生在两个查询观察点之间，即该读方法执行期间。八次拒绝同时证明私有暂存没有在版本变化后污染调用方。推断仍没有确定提交的纳秒时刻，也没有证明它与任何特定 WRITE 的硬件在途阶段相交。

执行证据包含启动前后二进制、运行服务端 PID/映像/config、两端 RXE 拓扑和 20 份实际源码快照；构建快照逐文件核对 SHA-256 与 Git blob。第二物理节点重算的审计 JSON 与主结果逐字节一致，篡改快照负控被拒绝。该单轮证据补充此前 SDK 时间区间实验，样本数不合并；20 次读取不能用于估计故障概率、性能收益或覆盖所有并发交错。

源码与证据：[S22] [S23]

20 部署与验证路线：先可复现契约，再判断硬件和业务收益

复现首先选定 Linux 客户端与服务端，明确设备、port、GID、控制端点、Redis 和存储目录。项目 Python 包要求 3.10 及以上；Rust 服务、SDK 与 FFI 通过根目录 make build 分别构建，部署配置用 TOML。RDMA 功能依赖 libibverbs/provider 和相应开发工具，真实 RXE 还需可用内核模块与网络。演示准备脚本只复制已构建产物和探针，不自动创建 KVM、bridge、设备或 Redis。

记录环境后先执行正常写读，验证完整内容、条带身份与单/双轨 hash；再运行晚到 WRITE 和受控 CQ 恢复，并把每次输出写入新目录。不要复用旧收据目录来覆盖失败。launch/completed 清单绑定实际二进制、wrapper 与 probe；server PID、可执行映像和配置哈希在前后保持一致。仅有 Git HEAD 不能证明正在运行的 server 来自该 HEAD，服务端进程身份需要单独验证。

现阶段最有依据的交付主张有三项：对象完整性与版本检查后的私有发布，失败前后的 DMA 生命周期收敛，以及固定计划下直接暂存的较低软件开销。有限状态探索检验了 19,657 个抽象状态和六个弱化守卫反例，帮助检视必要条件；它只建模两轨及有限条带，不含完整预算和 provider 状态。下一步可以加入多条带、准入释放、清理失败和发布锁的细化关系。

物理性能验证应先确认两条真正 ACTIVE 的独立 HCA 路径，并构造确实由一条网络路径限制的工作负载；再加入两 QP/单 HCA 对照，固定对象、并发、磁盘源条件和代码。逐区块收集 HCA/link、CPU/IRQ、磁盘、NUMA/PCIe 与内存计数，判断收益究竟来自链路、软件并发还是供给侧变化。当前物理单 HCA 消融和 RXE 双路径不能替代这项验证。

业务与发布还有独立任务：在 vLLM/Dynamo 中固定命中分布测首 token、吞吐与 CPU/RSS；为 Python 设计有明确对象生命周期的多轨公共 ABI；对锁定 xxh3 实现与跨库兼容进行独立迁移设计；完成上游 review、材料附件、官方提交和主办方回执。本报告提供可核查的技术与复现材料，不把这些尚未取得的运行结果或外部状态写成已完成。

源码与证据：[S01] [S16] [S24] [S25] [S26]

附录 A：Linux 构建与演示前置检查

以下命令用于独立测试环境。编译和核心测试应在具备 Linux RDMA 依赖的服务器执行；先按部署位置准备目录与权限。仓库记录的 guest 安装脚本面向新建 Ubuntu 22.04 guest，会安装匹配当前内核的 extra modules、rdma-core、provider、诊断工具及 Redis。构建开发依赖还需 protobuf-compiler、libibverbs-dev、libclang-dev、pkg-config、cmake，以及可用 Rust/C 工具链和 protoc。

make build 默认启用服务端 io-uring、rdma、metrics，客户端 rdma，并构建 rdma-ffi；默认无 rdma 的 Rust SDK 路径仍可独立使用。构建前会打印 profile/features/protoc 来源。protoc 可显式指定，否则依次发现仓库工具、/opt/contextstore 工具和 PATH。只有修改协议时才需 make proto；Python proto 生成还需项目 proto extra。

复现机器应记录 uname、编译器版本、Python、provider、rdma link、IP/GID 和 memlock。设备枚举不等于链路可用；configured fabric 与真实 listener 也需核对。不要把 RXE 设备显示为空的 PCI/NUMA 信息补成物理 HCA。服务端 Redis 和实际数据目录必须先就绪，演示脚本不承担这些基础设施的安装。

构建依赖与项目入口（在准备好的 Linux 测试机上执行） / bash

```
sudo apt-get install protobuf-compiler libibverbs-dev libclang-dev pkg-config cmake
pip install -e '.[test]'
make build

make test-server
pytest tests/ -v
rdma link show
ip -br addr
ulimit -l
```

可选：新建 Ubuntu guest 的已保存准备脚本 / bash

```
bash kv-service/deploy/softroce-vm/install-guest-prereqs.sh
# Then configure the two links with the repository's setup-guest-paths.sh.
# Inspect its server/client role and paths before applying to another guest.
```

源码与证据：[S01] [S16] [S25] [S26] [S31] [S32]

附录 B：已记录双 RXE fixture 的关键配置

以下配置摘自 10 月 16 日冻结 fixture 的 server.toml。它展示实际字段名和配套关系，路径与地址只适用于该隔离环境。两个数据目录属于同一 guest 的虚拟盘，不能由目录数量推断独立 NVMe 带宽。memory_tier 的 128 MiB 配置也不能与某次实验额外设置的 RDMA slab override 混为一个变量。完整文件以保存的原件为准。

服务端先启动两个真实 listener，再公布 fabric-a 与 fabric-b；本地对应 rxe_c0/rxe_c1、port 1、GID 1。显式 route 的第三字段是 Placement 中的 advertised endpoint，第四字段才是实际 listener。只配置第一条即可沿用同一 API 做一轨对照。启用 checksum 后应重新写入对象，以确认 Placement 含有完整条带摘要。

fixture 的核心 TOML 字段 / toml

```
[api]
listen = "10.31.0.2:55151"
max_connections = 100

[cluster]
node_id = "cs-soft-roce-vm-server"
grpc_advertise = "10.31.0.2:55151"
rdma_advertise = "10.31.0.2:55153"
data_nodes = []

[[cluster.rdma_rails]]
fabric_id = "fabric-a"
listener_endpoint = "10.31.0.2:55153"

[[cluster.rdma_rails]]
fabric_id = "fabric-b"
listener_endpoint = "10.32.0.2:55154"

[storage]
devices = ["/home/railtest/data/rail0", "/home/railtest/data/rail1"]
data_subdir = "contextstore"
striping_threshold = 4194304
striping_chunk_size = 4194304
rdma_stream_chunk_size = 1048576
verify_stripe_checksums = true

[memory_tier]
capacity_mb = 128
slab_size_mb = 4
use_pinned_memory = false

[metadata]
redis_url = "redis://127.0.0.1:6388/"
redis_key_prefix = "contextstore:soft-roce-vm:"
redis_connect_timeout_ms = 1000
redis_command_timeout_ms = 1000
```

已存在对象的手工双轨读取 / bash

```
./target/release/cs-rail-read-bench \
  --environment soft-roce \
  --coordinator http://10.31.0.2:55151 \

  --namespace ffi --object-key railfuse-demo-v2 \
  --direct-staging \
  --rail 'r0,rxe_c0,10.31.0.2:55153,10.31.0.2:55153,1,1' \
  --rail 'r1,rxe_c1,10.31.0.2:55153,10.32.0.2:55154,1,1' \
  --warmup 0 --iterations 1
```

源码与证据：[S18] [S26] [S27]

附录 C：Rust 公共 API 的完整对象读取

下面示例按当前公开接口组合，展示高层发布入口。构建调用方时启用 contextstore-client-rs 的 rdma feature。地址、设备和目标容量取自已经准备好的 fixture；示例用初查尺寸分配缓冲区，高层入口仍会自己重新查询并检查容量，因此对象在两次外层调用间变大时会明确报错。这个片段用于说明接口，没有作为本报告新增的编译或运行验收。

长期运行应复用一个 Arc<RailReader> 以共享预算；不要为每个并发对象独立新建 Reader，绕过聚合限额。若使用自动能力发现，先取得 lookup.placement，再用 LocalRailPath 和 RailReader::discover_from_placement 构造。发现结果变化时重建并重新核验；如果应用手工保存 StagedRailRead，则还需承担二次身份检查和最后发布责任。

示例在高层读取之前的一次 Lookup 只用于预先分配容量。若对象在高层方法自己的初查前缩小，一次合法成功可返回小于容量的有效长度；调用方只能消费 destination[..bytes]，不能把未发布的缓冲区尾部解释为本次对象。

使用带二次 Lookup 的 SDK 入口（接口示例） / rust

```
use std::sync::Arc;
use contextstore_client_rs::KvClient;
use contextstore_client_rs::rail_read::{RailLimits, RailReader, RailRoute};

use contextstore_client_rs::rdma::RdmaClientConfig;

async fn read_object() -> anyhow::Result<()> {
    let mut client = KvClient::connect("http://10.31.0.2:55151".into()).await
        .map_err(|error| anyhow::anyhow!(error.to_string()))?;
    let lookup = client.lookup_object("ffi", "railfuse-demo-v2").await?
        .ok_or_else(|| anyhow::anyhow!("object not found"))?;
    let mut destination = vec![0xA5; usize::try_from(lookup.descriptor.size)?];
    let limits = RailLimits { direct_staging: true, ..RailLimits::default() };
    let routes = vec![
        RailRoute::new("r0", "10.31.0.2:55153",
            RdmaClientConfig::new("10.31.0.2:55153", "rxe_c0")
                .with_port(1).with_gid_index(1)),
        RailRoute::new("r1", "10.31.0.2:55153",
            RdmaClientConfig::new("10.32.0.2:55154", "rxe_c1")
                .with_port(1).with_gid_index(1)),
    ];
    let reader = Arc::new(RailReader::new(routes, limits)?);
    let read = client.read_multi_rail_into(
        reader.clone(), "ffi", "railfuse-demo-v2", &mut destination, None
    ).await?;
    if let Some(bytes) = read {
        assert!(bytes <= destination.len());
        let published = &destination[..bytes];
        // Only this prefix belongs to the successful read.
        assert_eq!(published.len(), bytes);
        // Consume destination only after the high-level method succeeds.
    }
    for rail in reader.snapshots() {
        assert_eq!(rail.inflight_requests, 0);
        assert_eq!(rail.inflight_bytes, 0);
        assert_eq!(rail.registered_bytes, 0);
```

```
    }
    Ok(())
}
```

源码与证据：[S04] [S05] [S09] [S28]

附录 D：Python 单连接的安全使用与边界

最简单的示例使用内置缓冲区并复制出 bytes，从而不向业务暴露长期借用指针。进入 with 时连接，退出时 close。对于外部注册内存，应遵照包装器文档使用对齐、锁页且生命周期明确的分配，先注册再传输，注销后才释放；示例不以普通 NumPy 分配假装满足这些前提。

namespace 与 object_key 的规范 key 使用 UTF-8 字节长度前缀。现场对象 namespace 为 ffi，对象名为 railfuse-demo-v2，因此 canonical key 为 3:ffirailfuse-demo-v2。复杂 key 建议通过现有 SDK/工具形成并核对，不要把字符数误当字节数。失败后不要消费已有部分字节；记录错误，connect 重建连接，再重新执行完整读取。

根目录 Cargo workspace 的构建产物位于 target/release，而自动库发现不保证搜索这个目录。运行示例前应从仓库根目录导出 CONTEXTSTORE_RDMA_LIB，或把它设为已验收动态库的绝对路径。

从仓库根目录显式指定本次构建的 FFI 动态库 / bash

```
export CONTEXTSTORE_RDMA_LIB="$PWD/target/release/libcontextstore_rdma_ffi.so"
```

内置接收区读取，成功后复制出独立 bytes（接口示例） / python

```
from contextstore.storage.rdma_client import RdmaClient

with RdmaClient(

    "10.31.0.2:55153", device="rxe_c0", port=1,
    gid_index=1, buf_size_mb=128, timeout=30.0,
) as client:
    key = "3:ffirailfuse-demo-v2"
    count = client.get(key)
    if count:
        payload = client.buffer_view(count)
        assert len(payload) == count
        # The application owns payload; do not retain a raw borrowed view.
```

源码与证据：[S14] [S15] [S18]

附录 E：演示、历史实验与审计复算入口

准备脚本要求已有 Linux release FFI 和 cs-rail-read-bench。它复制当前包装器和探针、提取隔离旧库，并编译 test-only CQ observer。普通 wrapper 拒绝旧库，只有显式 negative probe 使用 unsafe-baseline。每个演示命令需要全新输出目录；异常负对照预期失败由驱动验证，不要把 baseline.exit=1 改写成零。

历史性能复现先用相同确定性对象种子准备 paper640/__combined__ 与 paper256direct0/__combined__，再在隔离 client guest 执行 five-arm driver。该脚本对尺寸串行运行，避免互相争用 CPU/磁盘；完整采样应保

留二进制、拓扑、服务端 source 路径和配置。只有重新采集满足同样身份/覆盖/模式检查的原始日志，才能声称复现性能结果。

审计已有日志与重新跑实验是两种操作。audit_rail_modes.py 可复算五组原始数据，audit_revision_ab.py 复算物理单轨版本消融，paper_pair_audit.py 复算修正基线后的 RXE 矩阵。Generation 见证以独立主审计和第二节点结果比对为准。所有这些历史材料的日期不随 PDF 重新生成而改变。

准备与分别执行已记录演示 fixture / bash

```
bash kv-service/verification/results/competition-demo-20261016-v2/prepare_demo.sh \
  "$PWD" /home/railtest/railfuse-demo-20261016-v2
cd /home/railtest/railfuse-demo-20261016-v2

python3 run_demo.py read --output receipts-read-01
python3 run_demo.py late --output receipts-late-01
python3 run_demo.py put --output receipts-put-01
python3 record_terminal.py --cast presenter-run.cast -- \
  python3 -u run_demo.py all --output receipts-all-01
```

在已准备相同对象的隔离 guest 上重新采集五组实验 / bash

```
COLLECTOR="$PWD/kv-service/benchmarks/collect_rail_modes.py" \
  bash kv-service/benchmarks/run_rxe_five_arm_20261009.sh \
  "$PWD/target/release/cs-rail-read-bench" /home/railtest/new-five-arm-results
```

复算已保存物理一轨与修正 RXE 基线日志 / bash

```
python3 kv-service/benchmarks/audit_revision_ab.py \
  --results kv-service/benchmarks/results/paper-fastbaseline-hca-2026-10-03 \
  --output /tmp/railfuse-recomputed-physical.json

python3 kv-service/benchmarks/paper_pair_audit.py \
  --results kv-service/benchmarks/results/paperfast-rxe-2026-10-03 \
  --prefix paperfast-sequential --prefix paperfast-c4 --prefix paperfast-c2 \
  --output /tmp/railfuse-recomputed-rxe.json
```

源码与证据：[S11] [S16] [S18] [S19] [S29] [S30]

附录 F：证据分层与阅读索引

证据阅读顺序建议为：先查源码契约与公开接口，再查实验方法和原始 audit，最后查 frozen/live 收据及对应命令。图像只承担机制解释；统计图来自 JSON。视频若缩短等待或剪辑场景，应保留未编辑 stdout、事件流与实际退出码，便于评审回到原始执行。

历史 checksum 使用 Cargo.lock 锁定的 twox-hash 1.6.3 实现。64 MiB index%251 fixture 在该 Rust 实现下为 a0a4cbfa5cad46af，独立 payload SHA-256 为 98dc891b284e4d84ac25b0c0a24fdbe39a7f0dbd643ad5e8aa06e02fc6258254。现代 Python xxhash 的标准化 XXH3 可能不同；本交付没有迁移持久化摘要，也没有宣称跨库互操作。

下面的状态表是主张边界，而不是额外功能列表。绿色本地测试、分支可访问、材料发布和官方收件分别需要自己的凭证。后续更新应追加新日期和新收据，保留旧失败、负结果和被替代的弱证据，以免材料失去可追溯性。

当前主张与验证状态

主张	已有依据	仍缺的证据
Rust 原子发布与故障收敛	源码、Mock、真实 RXE 故障与版本见证	所有 provider 与所有交错的实现证明
直接暂存减少软件开销	固定计划五组、每尺寸 10 区块与 CPU/RSS	更多硬件、尺寸和并发的稳健复现
Python FFI 复用与重连安全	旧库精确污染负控、8 GET + 2 CQ PUT 候选例	无注入自然 timeout、更多硬件 provider
物理多 HCA 聚合	有路径模型；有单 HCA 消融	双 ACTIVE 物理路径、同 NIC 两 QP 对照与瓶颈计数
推理业务提速	有 Connector 集成位置	固定模型与命中分布的端到端运行结果
形式化正确性	19,657 有限抽象状态与六个弱化反例	模型到实现 refinement 与更完整状态空间
上游与竞赛外部完成	可追溯源码与可审阅材料	合并、正式提交、主办方确认各自凭证

源码与证据：[S16] [S18] [S24]

源码与证据索引

源码及实验输入链接固定到 V3 制作前的 2705d2e 提交，便于评委核对当前叙述。目录链接指向相同提交的树；最新 PDF、视频入口由仓库首页维护。

S01 仓库结构、构建隔离、Scheduler/Worker 与层次访问规范
CLAUDE.md

S02 当前故事与历史证据的统一索引
docs/story.md

S03 历史输入日期、核心提交与未改运行时声明
kv-service/verification/results/competition-demo-20261016-v2/source-map.json

S04 lookup_object 与 read_multi_rail_into 高层流程
kv-service/client-rs/src/lib.rs

S05 数据结构、规划、预算、暂存、发布与 VerbsTransport 实现
kv-service/client-rs/src/rail_read.rs

S06 ObjectDescriptor、PlacementDescriptor 与 RdmaRailEndpoint 字段
kv-service/proto/kv_service.proto

S07 能力发现与旧新版本兼容契约
docs/rail-capability-discovery.md

S08 Mock、直接暂存、预算、取消与单轨 handoff 测试
kv-service/client-rs/src/rail_read/tests.rs

S09 原生客户端、RegisteredBuffer、QP/MR 释放顺序
kv-service/client-rs/src/rdma.rs

S10 命令行配置、实际模式与对象/每轨指标
kv-service/client-rs/src/bin/rail_read_bench.rs

S11 固定调度五组实验、资源口径与 RXE 限制
docs/paper/direct-staging-eval-2026-10-09.md

S12 源/目的内存生命周期与服务端 fallback 契约；早期性能状态按日期阅读
docs/multi-rail-design.md

S13 安全前提与验证范围
docs/paper/safety-contract.md

S14 C ABI 生命周期、错误前退休、CQ drain 与 fail-stop
kv-service/rdma-ffi/src/lib.rs

S15 Python 锁、ABI 拒绝、超时、指针和外部缓冲区责任
src/contextstore/storage/rdma_client.py

S16 冻结验收 165/11、13、6 与 12 个真实 RXE 用例
kv-service/verification/results/ffi-lifecycle-20261016/README.md

S17 同注册区、relay 时序与真实终态 late-WRITE 探针
kv-service/verification/ffi_rxe_late_write.py

S18 V2 新演示 launch/completed/seed/baseline/fixed/put-cq 及单/双轨日志
kv-service/verification/results/competition-demo-20261016-v2/live/receipts-take1/

S19 物理单轨代码消融、修正一轨基线后的 RXE 负结果
docs/paper/baseline-ablation-results.md

S20 64 MiB 原始审计的已归档精确数值
kv-service/verification/results/competition-demo-20261016-v2/assets/perf-64.json

S21 256 MiB 原始审计的已归档精确数值
kv-service/verification/results/competition-demo-20261016-v2/assets/perf-256.json

S22 前后 Generation 增长见证的环境、方法与推断边界
docs/paper/generation-transition-witness-2026-10-12.md

S23 20 reads / 15 overwrites / 10 accepted / 8 transition / 2 stale 的归档审计
kv-service/verification/results/competition-demo-20261016-v2/assets/generation-witness.json

S24 19,657 状态、六个负控与抽象模型假设
docs/paper/safety-model-plan.md

S25 Ubuntu guest RXE/provider/Redis 前置安装
kv-service/deploy/softroce-vm/install-guest-prereqs.sh

S26 Python 版本、依赖、make build/deploy、测试与当前贡献边界
README.md

S27 冻结双 RXE 服务端配置原件
kv-service/verification/results/ffi-lifecycle-20261016/server.toml

S28 crate 名称、rdma feature 与公开构建方式
kv-service/client-rs/Cargo.toml

S29 固定 10 block、5 iterations、1 warmup 的采集入口
kv-service/benchmarks/run_rxe_five_arm_20261009.sh

S30 演示产物准备与 CQ observer 构建

kv-service/verification/results/competition-demo-20261016-v2/prepare_demo.sh

S31 仓库记录的 Linux Rust/RDMA 构建依赖

kv-service/deploy/docker/Dockerfile

S32 Python >=3.10 与 test/proto extras

pyproject.toml